MLE-STAR Code Changes

Before / After the OpenAI-compatible reconstruction

Branch: feature/reconstruct-mle-star-openai
Base branch: feature/issue-1.5-openai-port
Generated: 2026-06-17

1. Configuration defaults

1.1 Conservative config values

The base branch used values suitable for a minimal demo (one candidate, no loops, checkers off). The reconstruction uses the conservative values from the reconstruction plan so the full pipeline can run end-to-end without excessive token cost.

BEFORE

```
agent_model = os.environ.get("ROOT_AGENT_MODEL", "gemini-2.0-flash-001")
num_model_candidates = 1
max_retry = 1
max_debug_round = 1
inner_loop_round = 0
outer_loop_round = 0
ensemble_loop_round = 0
use_data_leakage_checker = False
use_data_usage_checker = False
```

AFTER

```
agent_model = os.environ.get(
    "ROOT_AGENT_MODEL", "meta-llama-3.1-8b-instruct"
)
num_model_candidates = 2
max_retry = 2
max_debug_round = 2
inner_loop_round = 1
outer_loop_round = 1
ensemble_loop_round = 1
use_data_leakage_checker = True
use_data_usage_checker = True
```

2. Top-level pipeline wiring

2.1 agent.py run_pipeline

The base branch only ran initialization and submission. The reconstruction wires all four phases: initialization, refinement, ensemble, and submission.

BEFORE

```
from machine_learning_engineering.sub_agents.initialization import (
    agent as initialization_agent_module,
)
from machine_learning_engineering.sub_agents.submission import (
    agent as submission_agent_module,
)

initialization_agent_module.run_initialization_pipeline(state)
submission_agent_module.run_submission_pipeline(state)
```

AFTER

```
from machine_learning_engineering.sub_agents.ensemble import (
    agent as ensemble_agent_module,
)
from machine_learning_engineering.sub_agents.initialization import (
    agent as initialization_agent_module,
)
from machine_learning_engineering.sub_agents.refinement import (
    agent as refinement_agent_module,
)
from machine_learning_engineering.sub_agents.submission import (
    agent as submission_agent_module,
)

initialization_agent_module.run_initialization_pipeline(state)
refinement_agent_module.run_refinement_pipeline(state)
ensemble_agent_module.run_ensemble_pipeline(state)
submission_agent_module.run_submission_pipeline(state)
```

3. Initialization pipeline

3.1 Model candidate parser

The base parser assumed perfect JSON. If the small model returned extra text, the candidate list was empty and the pipeline stopped. The new parser tries JSON, then markdown code blocks, then falls back to hard-coded scikit-learn defaults.

BEFORE

```
response_text = common_util.get_text_from_response(response)
start_idx, end_idx = response_text.find("["), response_text.rfind("]") + 1
result = response_text[start_idx:end_idx]
models = ast.literal_eval(result)[:num_model_candidates]
for j, model in enumerate(models):
    ...
state[f"init_{task_id}_model_finish"] = True
except Exception:
    state[f"init_{task_id}_model_finish"] = False
```

AFTER

```
_DEFAULT_CANDIDATES = [
    {"model_name": "RandomForestRegressor", ...},
    {"model_name": "GradientBoostingRegressor", ...},
]

def _parse_model_candidates(response_text, num):
    start_idx, end_idx = text.find("["), text.rfind("]") + 1
    if start_idx != -1 and end_idx > start_idx:
        try:
            models = ast.literal_eval(text[start_idx:end_idx])
            if isinstance(models, list):
                return models[:num]
        except Exception:
            pass
    code_blocks = re.findall(r"```python\n(.*?)\n```", text, re.DOTALL)
    models = []
    for block in code_blocks[:num]:
        name_match = re.search(r"from sklearn\.\w+ import (\w+)", block)
        name = name_match.group(1) if name_match else "Model"
        models.append({"model_name": name, "example_code": block})
    return models if models else _DEFAULT_CANDIDATES[:num]
```

3.2 Initialization pipeline steps

The base pipeline retrieved one model, evaluated one model, and selected it. The new pipeline evaluates all candidates, merges the best with the rest, selects the best merged solution, and checks data usage.

BEFORE

```
# Retrieve/propose model (no web search in MVP)
run_agent(..., model_retriever_agent_1, ...)

# Generate and execute initial code with minimal debug
debug_util.run_and_debug(
    state, f"model_eval_{task_id}_1", ...)

# Select best solution
rank_candidate_solutions(state, task_id)
select_best_solution(state, task_id)
```

AFTER

```
run_agent(..., model_retriever_agent_1, ...)

# Evaluate every proposed model candidate
run_model_eval_for_all_candidates(state, task_id)

# Rank candidates and select the best base solution
rank_candidate_solutions(state, task_id)

# Merge the best candidate with the remaining candidates
run_merger_and_debug_loop(state, task_id)

# Promote the best merged (or base) solution
select_best_solution(state, task_id)

# Ensure all provided information is used
run_check_data_use_and_debug_loop(state, task_id)
```

3.3 Selection fallback

The original select_best_solution blindly promoted the selected merged solution. If the merger produced broken code, train_code_0_1 was broken too. The new version falls back to the best individual candidate.

BEFORE

```
best_idx = state.get(f"best_idx_{task_id}", 0)
response = state.get(f"merger_code_{task_id}_{best_idx}", "")
result_dict = state.get(f"merger_code_exec_result_{task_id}_{best_idx}", {})
code_text = response.replace("```python", "").replace("```", "")
with open(output_filepath, "w", ...) as f:
    f.write(code_text)
state[f"train_code_0_{task_id}"] = code_text
state[f"train_code_exec_result_0_{task_id}"] = result_dict
```

AFTER

```
best_idx = state.get(f"best_idx_{task_id}", 0)
response = state.get(f"merger_code_{task_id}_{best_idx}", "")
result_dict = state.get(f"merger_code_exec_result_{task_id}_{best_idx}", {})

if not result_dict or result_dict.get("returncode", 1) != 0:
    performance_results = state.get(f"performance_results_{task_id}", [])
    if performance_results:
        response = performance_results[0][1]
        result_dict = performance_results[0][2]

code_text = response.replace("```python", "").replace("```", "")
with open(output_filepath, "w", ...) as f:
    f.write(code_text)
state[f"train_code_0_{task_id}"] = code_text
state[f"train_code_exec_result_0_{task_id}"] = result_dict
state["best_initial_code"] = code_text
state["best_initial_score"] = result_dict.get("score")
```

3.4 MODEL_RETRIEVAL_INSTR prompt

The original prompt asked for one candidate and used escaped double braces in the example, which confused the small model. The new prompt asks for num_model_candidates and uses cleaner formatting.

BEFORE

```
- Propose {num_model_candidates} simple and effective machine learning model...
Use this exact JSON format (and nothing else):
[
  {{"model_name": "Name", "example_code": "import pandas..."}}
]
```

AFTER

```
- Propose {num_model_candidates} simple and effective machine learning models...
Use this exact JSON format (and nothing else):
[
  {"model_name": "Name", "example_code": "from sklearn..."}
]
```

3.5 CODE_INTEGRATION_INSTR prompt

The merger prompt did not forbid loading test.csv. The small model sometimes generated ensemble code that made predictions on the test set, causing feature-name mismatches. The new prompt explicitly forbids it.

BEFORE

```
- Only use the provided train data in the `./input` directory.

# Required
- Print out ... 'Final Validation Performance: ...'
```

AFTER

```
- Only use the provided train data in the `./input` directory.
  Do NOT load test.csv for the validation score.

# Required
- Print out ... 'Final Validation Performance: ...'
```

4. Code execution utilities

4.1 extract_performance_from_text

The original parser required the exact format 'Final Validation Performance: <number>'. The small model often added formatting. The new parser extracts the last number from the performance line.

BEFORE

```
for line in lines:
    if "Final Validation Performance" in line:
        parts = line.split(":")
        score_str = parts[-1].strip()
        performance_value = float(score_str)
```

AFTER

```
import re

for line in lines:
    if "Final Validation Performance" in line:
        matches = re.findall(
            r"[-+]?\d*\.?\d+(?:[eE][-+]?\d+)?", line)
        if matches:
            return float(matches[-1])
```

4.2 debug_util run_and_debug with leakage check

The base run_and_debug stopped after a successful execution. The new version runs the data-leakage checker when use_data_leakage_checker is True.

BEFORE

```
result = _get_result(state, agent_name)
if _is_successful(result):
    return

# Debug loop ...
```

AFTER

```
result = _get_result(state, agent_name)
if _is_successful(result):
    check_leakage_util.check_and_fix_leakage(
        state, agent_name,
        max_iterations=state.get("max_retry", 1),
    )
    return

# Debug loop ...
```

4.3 debug_util plan_implement handling

The original logic assumed the LLM returned only the replacement block. It tried prev_code.replace(code_block, code). The small model returned the full script, so the replacement often broke the code. The new logic stores the full response as the improved solution.

BEFORE

```
elif agent_name.startswith("plan_implement"):
    if "debug" not in agent_name:
        task_id = agent_name.split("_")[-1]
        step = state.get(f"refine_step_{task_id}", 0)
        code_block = state.get(f"refine_code_block_{step}_{task_id}", "")
        prev_code = state.get(f"train_code_{step}_{task_id}", "")
        new_code = prev_code.replace(code_block, code)
    else:
        new_code = code
```

AFTER

```
elif agent_name.startswith("plan_implement"):
    # The LLM returns the complete improved solution.
    new_code = code
    if "debug" not in agent_name:
        task_id = agent_name.split("_")[-1]
        step = state.get(f"refine_step_{task_id}", 0)
        inner_iter = state.get(f"inner_iter_{task_id}", 0)
        state[f"train_code_{step}_{task_id}"] = new_code
        state[f"train_code_improve_{inner_iter}_{step}_{task_id}"] = new_code
```

5. Data leakage checker rewrite

5.1 check_leakage_util.py architecture

The original file built a SequentialAgent using google.adk classes. That cannot run in the OpenAI port. The new file is a plain Python module that calls run_agent directly and manipulates AgentState.

BEFORE

```
from google.adk import agents
from google.adk.agents import callback_context as callback_context_module
from google.genai import types

def get_data_leakage_checker_agent(prefix, suffix):
    check_leakage_agent = agents.Agent(...,
        before_model_callback=..., after_model_callback=...)
    ...
    return agents.SequentialAgent(...)
```

AFTER

```
from machine_learning_engineering.runner import run_agent

def _check_leakage_status(state, agent_name):
    code = _get_code_for_agent(state, agent_name)
    instruction = CHECK_LEAKAGE_INSTR.format(code=code)
    response = run_agent(state, f"check_leakage_agent_{agent_name}", ...)
    return _parse_leakage_response(response_text)

def check_and_fix_leakage(state, agent_name, max_iterations=2):
    if not state.get("use_data_leakage_checker", False):
        return True
    for _ in range(max_iterations):
        ...
```

6. Submission pipeline

6.1 _create_submission_workspace

The original function deleted the ensemble workspace every time. That destroyed files created by the ensemble agent. The new function preserves the workspace if it already contains data.

BEFORE

```
run_cwd = os.path.join(workspace_dir, task_name, "ensemble")
if os.path.exists(run_cwd):
    shutil.rmtree(run_cwd)
os.makedirs(run_cwd, exist_ok=True)
os.makedirs(..., exist_ok=True)
# copy all input files
```

AFTER

```
run_cwd = os.path.join(workspace_dir, task_name, "ensemble")
os.makedirs(run_cwd, exist_ok=True)
os.makedirs(..., exist_ok=True)
if os.listdir(os.path.join(run_cwd, "input")):
    return
# copy all input files only if needed
```

6.2 get_submission_and_debug_agent_instruction

The base version always used train_code_0_1. The new version selects the best among refined, ensemble, and initial solutions.

BEFORE

```
# In the MVP the best solution is always train_code_0_1.
final_solution = state.get("train_code_0_1", "")
return prompt.ADD_TEST_FINAL_INSTR.format(...)
```

AFTER

```
best_ensemble_code = state.get("best_ensemble_code", "")
best_ensemble_score = state.get("best_ensemble_score")
refined_score = state.get("best_refined_score")
refined_code = state.get("best_refined_code", "")
initial_code = state.get("best_initial_code", refined_code)

best_code = refined_code
best_score = refined_score
if best_ensemble_code and best_ensemble_score is not None:
    ...
if not best_code or best_score is None or best_score == 1e9:
    best_code = initial_code
return prompt.ADD_TEST_FINAL_INSTR.format(...)
```

7. New modules

7.1 refinement/agent.py (new file)

This file did not exist in the base branch. It implements the full refinement loop: ablation, summary, plan, implement, and inner/outer loops. Below is the core pipeline function:

```
def run_refinement_pipeline(state) -> None:
    num_solutions = state.get("num_solutions", 1)
    outer_loop_round = state.get("outer_loop_round", 0)
    inner_loop_round = state.get("inner_loop_round", 0)

    for task_id in range(1, num_solutions + 1):
        init_code = state.get(f"train_code_0_{{task_id}}", "")
        _set_current_solution(state, task_id, 0, init_code)
        ...
        for step in range(outer_loop_round):
            state[f"refine_step_{{task_id}}"] = step
            run_ablation_and_debug_loop(state, task_id)
            run_ablation_summary_agent(state, task_id)
            run_init_plan_agent(state, task_id)
            run_init_plan_implement_agent(state, task_id)
            for inner_iter in range(1, inner_loop_round + 1):
                run_plan_refine_agent(state, task_id, inner_iter)
                run_plan_implement_agent(state, task_id, inner_iter)
            _select_best_improvement(state, task_id, step, inner_iters)
```

7.2 ensemble/agent.py (new file)

This file also did not exist in the base branch. It builds an ensemble workspace, generates ensemble plans, implements them, and selects the best one.

```
def run_ensemble_pipeline(state) -> None:
    ensemble_loop_round = state.get("ensemble_loop_round", 0)
    if ensemble_loop_round < 1:
        return
    _create_ensemble_workspace(state)
    run_init_ensemble_plan_agent(state)
    run_init_ensemble_plan_implement_agent(state)
    for ensemble_iter in range(1, ensemble_loop_round + 1):
        run_ensemble_plan_refine_agent(state, ensemble_iter)
        run_ensemble_plan_implement_agent(state, ensemble_iter)
    _select_best_ensemble(state)
```

8. Summary of changes

The reconstruction made the following types of changes:
- Replaced Google ADK agents with direct calls to the OpenAI-compatible runner.
- Added robust parsing and fallback defaults for the small model.
- Extended the initialization pipeline to support multiple candidates, merging, data-use checking, and leakage checking.
- Added entirely new refinement and ensemble modules.
- Strengthened the submission agent with best-solution selection and workspace preservation.
- Updated prompts to forbid common failure modes (test.csv usage, hallucinated columns).
- Added fallback mechanisms so the pipeline always produces a submission even when some agents fail.